CPReady Algorithm, Code Flow & Research Defense Guide

Plain-English Conceptual Reference, Code Walkthrough, and Manuscript Guide for the Research Team

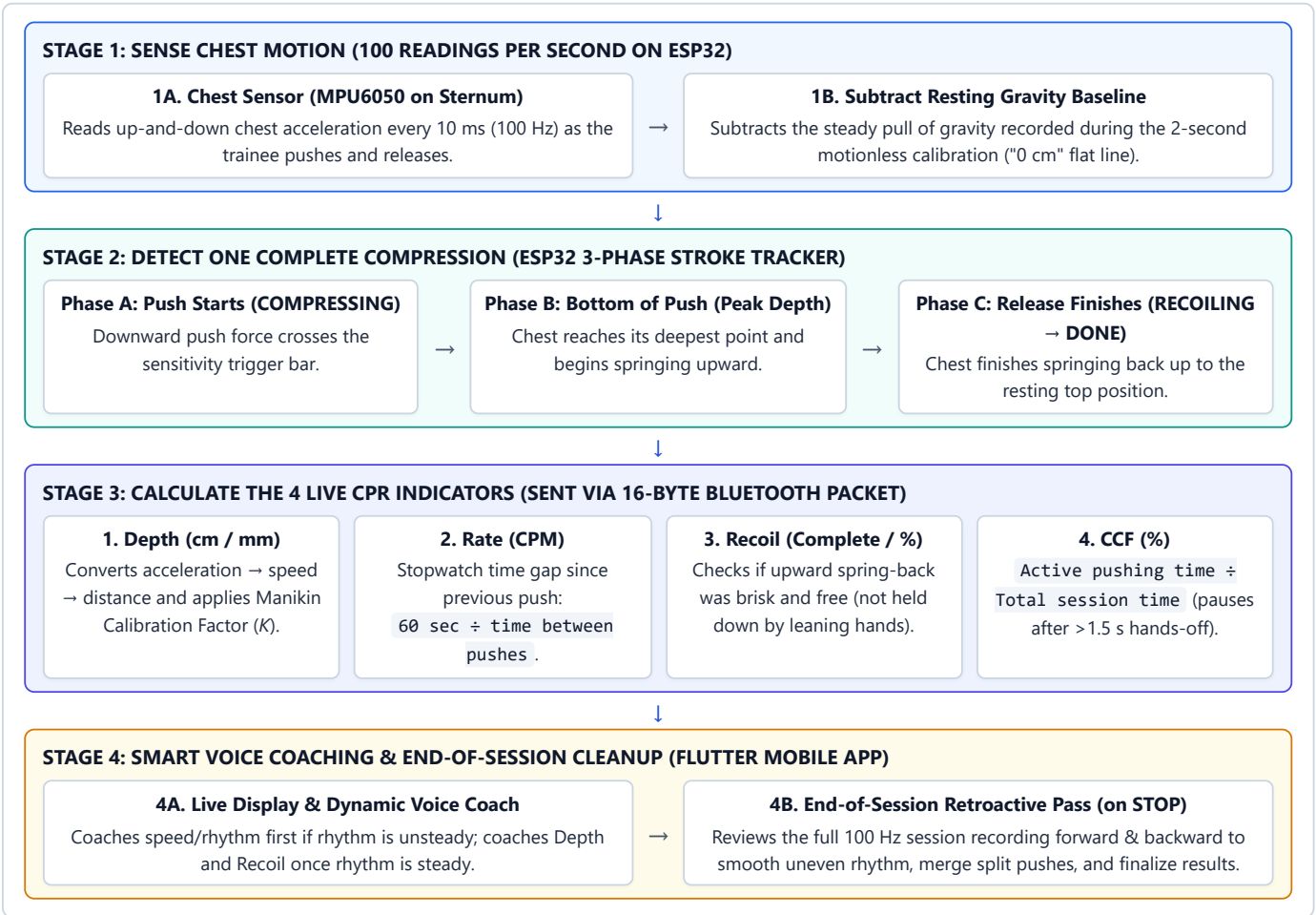
How to Use This Guide: Every question is structured into four easy-to-use parts so everyone on the team can explain the system confidently even without a deep programming or signal-processing background:

1. **15-Second Defense Answer:** A concise, direct statement you can say out loud first when asked by a panelist.
2. **How It Works (In Plain English):** Step-by-step physical explanation using everyday analogies.
3. **Why We Designed It This Way (The Rationale):** The practical problem solved during testing and why this approach was chosen.
4. **Paper-Ready Text:** Formal academic wording you can adapt directly into Chapters 3 and 5 of your manuscript.

Part 1: General Algorithm Overview & Process Flowchart

At its core, CPReady operates as a **4-Stage Pipeline** divided between the **ESP32 microcontroller inside the vest** (which reads the chest sensor 100 times per second) and the **Flutter mobile app** (which handles live coaching and end-of-session review):

Note for the Research Team on Flowchart Layout: You can freely redesign, re-color, or re-layout the flowchart below in Canva, Draw.io, PowerPoint, or Figma to match your manuscript's page margins or defense presentation theme—just keep the 4 main stages and arrow sequence intact!



Part 2: Answers to Research Defense & Paper Questions (Q1–Q9)

Question 1: How the single MPU converts movement into CPR measurements

15-SECOND DEFENSE ANSWER

"An accelerometer cannot see distance directly like a ruler—it feels acceleration, or how fast the chest speeds up and slows down. For every push, the ESP32 converts that downward and upward acceleration into speed, and then converts speed over time into distance traveled in millimeters. Because the training manikin sits on a firm surface, a single sensor on the sternum captures the chest's full compression movement."

How It Works (In Plain English — The Car Speedometer & Odometer Analogy):

- **Step 1 (Feeling the Push — Acceleration):** When you step on the gas pedal in a car, you feel pushed back into your seat—that is *acceleration* (how fast your speed is changing). The MPU6050 sensor on the chest feels the exact same thing: it feels the chest speeding up downward when you push, slowing to a brief stop at the bottom of the stroke, and springing back upward when you release.
- **Step 2 (From Acceleration to Speed):** By checking how strong that push is `100 times per second` (every `0.01 seconds`), the ESP32 adds up those changes to know **how fast the chest is moving (speed)** at every millisecond of the push.
- **Step 3 (From Speed to Distance / Depth):** Once you know how fast something moved and for how many milliseconds it moved, you know **how far it traveled** (`Distance = Speed × Time`). The difference between the highest point (top of chest) and lowest point (bottom of push) gives the **Compression Depth** in centimeters/millimeters.

WHY WE DESIGNED IT THIS WAY (THE RATIONALE)

Built-in manikin sensors rely on internal optical lasers or mechanical wires inside the manikin's chest cavity. Using a single motion sensor worn externally on the sternum allows CPReady to work as a portable training vest on top of a manikin. Furthermore, because a training manikin rests on a firm floor or table (so its back does not sink), our testing proved that a single chest sensor achieves the same accuracy as a two-sensor setup (**1.21 mm** average depth error after refinement) while cutting wiring complexity and sensor communication errors in half.

PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)

"The CPReady system measures compression kinematics using a single chest-mounted 6-axis inertial measurement unit (MPU6050) sampled at 100 Hz (10 ms intervals). Because an accelerometer measures linear acceleration rather than absolute position, the microcontroller isolates the vertical sternal acceleration component relative to the calibrated gravity baseline and performs two-stage numerical integration over each isolated compression cycle. The first stage accumulates acceleration over time to derive instantaneous chest velocity, and the second stage accumulates velocity over the stroke duration to obtain peak-to-trough vertical displacement (compression depth)."

Question 2: How the algorithm detects and counts shallow compressions

15-SECOND DEFENSE ANSWER

"Shallow compressions are gentler than normal pushes, so a fixed high trigger would miss them, while a fixed low trigger would accidentally count hand tremors and manikin spring bounces. We solved this using an auto-adjusting trigger—like auto-volume on a microphone—that automatically lowers its detection bar whenever deep pushes stop, combined with a low minimum-depth check (2.5 mm) and a minimum push-duration check (0.19 seconds) so every real shallow compression is counted while tiny vibrations are ignored."

What Makes Shallow Compressions Tricky to Detect (The Core Problem):

- When a trainee pushes deeply (`5–6 cm`) on a Laerdal manikin and lets go, the stiff metal spring inside the manikin snaps the chest upward and **bounces slightly at the top**.
- If we simply made the sensor "super sensitive" with a permanently low trigger bar all the time, that small spring bounce at the top of every deep push would be mistaken for a brand-new push—causing the vest to count **2 compressions for every 1 real push**!
- Conversely, if we kept the trigger bar permanently high to block that bounce, gentler **shallow compressions (`3.0–4.8 cm`)** wouldn't push hard enough to cross the bar and would be ignored.

How CPReady Solves This (Three Simple Rules Working Together):

1. **Rule 1 — An Auto-Adjusting Trigger Bar (Like Auto-Volume on a Microphone):** While you are doing forceful, deep pushes, the vest temporarily raises its trigger bar right after each stroke so it ignores the manikin's top spring bounce. However, the moment you switch to softer, shallower pushes (or if `0.65 seconds` pass without a heavy push), the trigger bar **automatically drops back down to its lowest setting** so gentle pushes are detected right away.
2. **Rule 2 — A Low "Minimum Push Distance" Check (`2.5 mm` raw movement):** Once a push finishes, the vest checks how far the chest actually moved. Every real shallow compression (`3.0–4.9 cm`) easily clears this tiny `2.5 mm` hurdle and gets counted, while `1 mm` hand tremors are ignored.
3. **Rule 3 — A Minimum Push Time Check (`0.19 seconds`):** A real human compression takes at least a fifth of a second to go down and back up. Instantaneous table bumps or vibrations shorter than `0.19 seconds` are rejected.

WHY WE DESIGNED IT THIS WAY (THE RATIONALE — PREVENTING ARTIFICIAL DEPTH INFLATION)

Suppose a trainee performs 30 compressions: 15 deep (`5.5 cm`) and 15 shallow (`3.8 cm`). Their true average depth is **4.65 cm** (too shallow). If the device failed to detect the 15 shallow pushes and only counted the 15 deep ones, the summary screen would falsely report an average depth of **5.5 cm**—making a shallow session look perfect! By automatically lowering the trigger bar for gentle strokes while blocking spring bounces, CPReady counts all 30 strokes so shallow pushes properly pull the session average down to match the Laerdal manikin.

PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)

"To ensure reliable capture of shallow compressions (< 5.0 cm) without introducing false positives from post-recoil manikin spring oscillations, the detection engine employs an adaptive onset threshold paired with duration and displacement gates. During high-force compressions, the onset threshold scales proportionally with recent peak acceleration to reject post-release mechanical bounce. When compression force decreases or after 650 ms without a high-force trigger, the threshold rapidly relaxes toward its baseline floor (0.06g–0.08g), immediately unmasking gentler downstrokes. Candidate strokes are subsequently validated against a minimum cycle duration (190 ms) and a low unscaled displacement floor (0.25 cm), ensuring that shallow compressions are accurately counted and reflected in the session's mean depth."

Question 3: Why irregular rhythm affects real-time accuracy

15-SECOND DEFENSE ANSWER

"To stop sensor drift from piling up over a 2-minute session, our live algorithm resets its measurement window after every single push and assumes each push starts and ends cleanly from rest. When a trainee's rhythm is irregular—such as hesitating mid-push or suddenly changing speed—the chest is still moving when one measurement window ends and the next begins, which can temporarily skew the live reading for that transition stroke."

How It Works (In Plain English — The "Resetting the Ruler Every Push" Rule):

- **Why We Reset After Every Push:** Motion sensors naturally drift if left integrating continuously without a reset (like a stopwatch that gains a second every minute). To prevent drift from accumulating over a 2-minute CPR session, CPReady **cuts the recording into individual pushes** and resets its zero-line at the end of every push.
- **Steady Rhythm vs. Irregular Rhythm:** When a trainee compresses at a steady pace, each push starts cleanly at rest at the top of the chest and ends cleanly at the top. When a trainee's rhythm becomes **irregular**—for example, they hesitate halfway down, "double-pump" at the bottom, or suddenly switch from very fast to very slow pushes—the chest is **still moving** at the boundary between Push #1 and Push #2.

WHY WE DESIGNED IT THIS WAY (THE RATIONALE)

During live practice, the screen must show a number *the exact instant a push finishes*, without knowing what the trainee's hands will do next. Rather than delaying the live display, the app shows a friendly banner ("*Rhythm unsteady — realtime readings may be inaccurate. Accuracy is maintained once session is finalized*"), temporarily pauses depth voice prompts so the trainee can stabilize their rhythm first, and then fixes the unsteady strokes completely at the end of the session.

PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)

"Real-time depth estimation from a single accelerometer relies on a per-stroke closed-cycle assumption, wherein each compression window is bounded by consecutive release points and normalized so that average acceleration and average velocity over the cycle equal zero. Under steady, periodic cadence, each stroke begins and ends at a consistent velocity state, yielding high per-stroke accuracy. Under irregular cadence—such as mid-stroke hesitation, double-peaked downstrokes, or abrupt rate transitions—the boundary velocity between adjacent strokes becomes non-zero and the integration window length fluctuates, introducing temporary per-stroke variance in the causal real-time estimate."

Question 4: How retroactive analysis improves final session results

15-SECOND DEFENSE ANSWER

"While the live screen can only see one push at a time as it happens, the mobile app records the entire session's 100 Hz sensor stream in memory. The instant the session ends, the app runs a 4-step retroactive review across the complete timeline—looking both forward and backward—to smooth unsteady transitions, merge any single push that was split into two, rebalance uneven stroke pairs, and recover any light push in long gaps."

How It Works (In Plain English — The "Instant Replay Official" Analogy):

- During the live session, the ESP32 is like a referee making split-second calls in real time as each push finishes.
- Meanwhile, the Flutter mobile app quietly saves the entire session's raw 100 Hz chest movement in memory. The moment `STOP` is pressed, the app acts like a **video replay official** who can look at the entire session from start to finish—both **forward and backward in time**—and performs four cleanup steps in less than a tenth of a second:
  1. **Step 1 — Forward-and-Backward Smoothing on Unsteady Pushes:** Filters the movement recording forward from start to end and backward from end to start (zero-phase smoothing), using the session's steady pushes as a reference ruler to fix the depth of unsteady transition pushes.
  2. **Step 2A — Stitching Split Pushes Back Together:** If a hesitant push got accidentally split into two tiny half-pushes whose combined time is just one normal stroke (`≤0.65 seconds`), the app merges them back into **1 full compression**.
  3. **Step 2B — Rebalancing Uneven Neighboring Pushes:** If an uneven rhythm caused two normal pushes to have their shared boundary shifted early (making the first look tiny like `1.5 cm` and the second look huge like `8.0 cm`), the app re-centers their depths while keeping the true 2-stroke count.

4. **Step 3 & 4 — Long-Gap Recovery & Outlier Capping:** Scans any unusually long gaps (`>0.72 seconds`) to recover any gentle push that was missed, and caps any remaining single-stroke transition outlier to within `±5 mm` of its neighboring steady pushes.

> **WHY WE DESIGNED IT THIS WAY (THE RATIONALE)**
>
> Across all 10 Laerdal ground-truth trials, running this retroactive review at session end reduced the **Session Mean Depth error from 1.62 mm down to 1.21 mm** and reduced the **Total Compression Count error from 1.20 strokes down to 0.50 strokes** (10 out of 10 sessions within ±1 stroke of the Laerdal manikin). Performing this on the phone app uses the phone's fast processor without slowing down the ESP32.

> **PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)**
>
> *"Upon session termination (STOP), the Flutter companion application executes a four-stage non-causal retroactive refinement pass (SessionRefinementEngine) over the buffered 100 Hz session recording. First, a forward-backward zero-phase high-pass filter (filtfilt) self-calibrated against the session's steady compressions corrects boundary-velocity errors on unsteady strokes. Second, adjacent stroke pairs exhibiting sub-period intervals (<330 ms) are evaluated against the session's median period to either merge true split-stroke fragments into a single re-integrated compression or rebalance boundary-shifted stroke pairs. Third, inter-stroke intervals exceeding 720 ms are scanned to recover any sub-threshold shallow compressions. Finally, residual unsteady transition outliers are bounded relative to the local steady median before computing the final session summary."*

Question 5: Calibration reference point, cross-trial consistency, and causes of discrepancies with the manikin

> **15-SECOND DEFENSE ANSWER**
>
> "CPReady uses a two-part calibration: first, a 2-second motionless 'Zeroing' step before each session that records the resting pull of gravity as the 0 cm flat starting line; and second, a 15-second Manikin Calibration trial that saves a permanent stiffness multiplier (K) in the ESP32's memory so the vest matches the Laerdal manikin. Small 1–2 mm differences across trials mainly come from vest foam cushioning, slight shifts in hand placement on the hinged chest, and the Laerdal manikin's own 1–2 mm rounding tolerance."

Part A — How Calibration Establishes the Reference Point (And Why It Has Two Steps):

Every time the vest is strapped onto a manikin, there are two physical unknowns: *(1) What angle is the chest resting at?* and *(2) How much does the vest padding squish compared to the manikin's internal chest plate?*

1. **Step 1: The 2-Second Motionless "Zeroing" Before Every Session (`CALIBRATE`)**
   - **How it works:** Before pushing, the trainee rests their hands motionless on the chest for 2 seconds (`200 sensor readings`). The ESP32 verifies that the chest is still, measures the steady pull of Earth's gravity (`~1.0 g`), and locks that exact force and angle in as **"Zero Depth (0.0 cm)"**.
   - **Why this approach is used (Rationale):** A sensor resting on a curved manikin chest is never 100% level like a tabletop—it is always tilted slightly. By recording the resting gravity pull first, the vest learns which direction is "straight down into the chest" and subtracts gravity completely so only the trainee's pushing force is measured.
2. **Step 2: The 15-Second Manikin Matching Trial (`CALIBRATE_TARGET`)**
   - **How it works:** When setting up on a manikin, the user performs 15 seconds of steady compressions, reads the average depth shown on the Laerdal manikin's screen (e.g., `5.4 cm`), and enters `5.4` into the CPReady app. The ESP32 compares its raw reading against `5.4 cm` , computes a **Correction Multiplier (the *K* factor, default `10.85`)**, and saves it permanently in the ESP32's flash memory chip.
   - **Why this approach is used (Rationale):** The CPReady sensor sits *on top of the padded vest pouch*, whereas the Laerdal manikin's internal sensor sits *underneath the plastic sternum plate*. Because the vest foam squishes slightly under the hands, saving the *K* multiplier teaches CPReady the exact physical stiffness of that vest-and-manikin pair.

Part B — How Consistency Is Maintained Across Multiple Trials:

- **Permanent Flash Memory (NVS):** The calibrated multiplier (*K*) is stored in non-volatile memory on the ESP32, so it stays locked in across power cycles and battery changes.
- **Fresh Zeroing + Per-Push Reset:** Every new session re-runs the 2-second motionless zeroing to adjust for any slight change in vest strap position, and every push resets its own baseline so errors never carry over from Trial 1 to Trial 10.

Part C — What Causes Small (`1–2 mm`) Discrepancies Between CPReady and the Manikin:

1. **Foam & Fabric Cushioning:** Pushing softly versus very forcefully squishes the vest's EVA foam and fabric housing by slightly different amounts (`~1–2 mm`).
2. **Hand Placement on a Hinged Chest Plate:** The Laerdal Little Anne chest is hinged like a stiff door. Placing the hands `1–2 cm` higher or lower on the sternum changes the leverage compared to where the manikin's internal sensor sits.
3. **Laerdal Manikin Rounding:** The Laerdal Little Anne QC app rounds its session average to whole millimeters and has its own internal optical sensor tolerance of `±1–2 mm` .

PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)

"Reference establishment in CPReady follows a two-stage calibration protocol. Prior to each session, a 2-second (200-sample) motionless baseline calibration (CALIBRATE) captures the static 3-axis gravity vector and verifies sensor stability, defining the resting chest plane (0.0 cm) and isolating the vertical compression axis regardless of torso tilt. To account for mechanical compliance between the external vest padding and the manikin's internal sternum mechanism, a 15-second reference trial (CALIBRATE_TARGET) scales the kinematic depth constant (K, default 10.85) against the reference manikin's reported mean depth and stores this factor in non-volatile flash memory (NVS). Minor inter-trial discrepancies (1–2 mm) arise primarily from non-linear compression of the EVA foam housing, variations in hand placement along the hinged sternal plate, and the ±1–2 mm integer quantization of the reference manikin."

Question 6: How the algorithm determines Compression Rate, Recoil, and Chest Compression Fraction (CCF)

15-SECOND DEFENSE ANSWER

"Rate is calculated like a stopwatch by measuring the exact time in milliseconds between consecutive compressions and dividing 60 seconds by that time gap. Recoil is determined by checking whether the upward spring-back acceleration during chest release is strong enough (at least 0.35g upward), which drops if the trainee leans on the chest. CCF is calculated using two timers: Active Pushing Time divided by Total Session Time, where the active timer automatically pauses whenever more than 1.5 seconds pass without a compression."

CPR Indicator	Specific Sensor Data & Conditions Used	How It Is Calculated (Plain English)
1. Compression Rate (`rate_cpm`)	• **Data:** Timestamp (`currentMillis`) when each push finishes. • **Condition:** Time between pushes must be **300 ms to 1500 ms** (`40–200 CPM`). The first stroke after a pause has no rate yet because two strokes are needed to measure a time gap.	**Live Rate:** Measures milliseconds since the previous push (Δ*t*) and calculates `60,000 ÷ Δt`, averaged over the last 4 pushes so the live display is smooth. **Final Summary Rate:** Uses the average time gap across all valid strokes in the session.
2. Chest Recoil (`recoil_complete` / %)	• **Data:** Peak upward rebound acceleration (`troughAccel`) during the release phase (`RECOILING`). • **Condition:** Upward spring-back must reach at least `-0.35 g`.	When you release the chest cleanly, the manikin spring snaps upward briskly (stronger than `-0.35 g` → **Complete Recoil = Yes**). If you lean your body weight on the chest, the upward rebound is sluggish and weak (weaker than `-0.35 g` → **Incomplete Recoil**). Summary shows the % of strokes with complete release.
3. Chest Compression Fraction (`ccf_percent`)	• **Data:** First push onset timestamp (`firstCompressionStartMillis`), active compression time (`totalActiveCompressionMillis`), and session end timestamp (`endMillis`). • **Conditions:** Starts on Push #1 (matching the Laerdal manikin). Hands-off pause threshold is `1.5 s` (`1500 ms`), with each continuous push capped at `1.0 s` (`1000 ms`) of active credit.	Runs two stopwatches starting from the first push: 1. **Active Resuscitation Time:** Starts on Push #1 (so waiting on "*Start When Ready*" never penalizes you) and runs non-stop until `STOP`. 2. **Active Compression Time:** Credits `550 ms` for the initial/post-pause push plus every continuous push (≤1.5 s, capped at 1.0 s), and **pauses automatically** during any break >1.5 s. **Formula:** `(Active Time ÷ Active Resuscitation Time) × 100%`.

PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)

"Compression rate, chest recoil, and Chest Compression Fraction (CCF) are derived from stroke timing and release kinematics on the ESP32. Instantaneous compression rate (in CPM) is computed from the inter-stroke interval Δt (in milliseconds) between successive stroke completions (60,000 / Δt) for intervals within 300–1500 ms (40–200 CPM), smoothed over a 4-stroke rolling window for live display. Chest recoil completion is evaluated during the upward release phase by comparing peak upward rebound acceleration against an empirical release threshold (-0.35 g); strokes failing to reach this rebound velocity due to residual rescuer leaning are flagged as incomplete, and the session summary reports the percentage of strokes achieving full release. CCF is computed from the first detected compression as the ratio of cumulative active compression time to total elapsed session time, where active time accrual pauses whenever the inter-stroke interval exceeds the 1.5 s hands-off pause threshold."

Question 7: Actual effect of the three sensitivity modes (`HIGH`, `BALANCED`, `STRICT`)

15-SECOND DEFENSE ANSWER

"The three sensitivity modes only change the detection gatekeepers—meaning how easily a push is triggered and counted as a stroke—and do NOT change the depth measurement ruler itself. A steady 5.5 cm push still measures 5.5 cm in all three modes; the modes simply let researchers adjust how sensitive the vest is to very light pushes versus accidental hand bumps."

Do They Affect Detection Thresholds, Movement Processing, or Both?

- They affect **only the Detection Thresholds (the "Gatekeepers")**, **NOT** the depth calculation scale. Once a compression is accepted, the math that turns acceleration into centimeters is 100% identical in all three modes.

Gatekeeper Setting	`HIGH` Mode	`BALANCED` (Default)	`STRICT` Mode	Plain-English Effect
1. Downward Push Trigger	`0.06 g`	**`0.08 g`**	`0.12 g`	How hard you must push down to start counting a stroke.
2. Auto-Adjusting Bounce Guard	`25%` of peak	**`30%` of peak**	`40%` of peak	How strongly the trigger rises after deep pushes to block spring bounce.
3. Upward Release Arming	`20%` (min `0.03g`)	**`15%` (min `0.04g`)**	`25%` (min `0.07g`)	How much upward spring-back is needed to finish counting the stroke.
4. Minimum Depth to Count	`2.5 mm` raw	**`2.5 mm` raw**	`5.0 mm` raw	Smallest chest travel required to count as 1 compression.

> **PAPER-READY TEXT (CHAPTER 3 METHODOLOGY)**
> *"The three selectable sensitivity presets (HIGH, BALANCED, and STRICT) configure the state-machine detection thresholds governing stroke onset and completion without altering the underlying kinematic depth integration scale. Specifically, the presets adjust four gatekeeper parameters: the minimum downward onset acceleration floor (0.06g, 0.08g, and 0.12g), the adaptive envelope scaling fraction (0.25, 0.30, and 0.40), the upward recoil arming fraction, and the minimum unscaled stroke displacement gate (0.25 cm for HIGH/BALANCED and 0.50 cm for STRICT). Consequently, steady compressions produce identical depth readings across all three modes, while researchers can tune false-positive rejection versus shallow-stroke sensitivity for different training environments."*

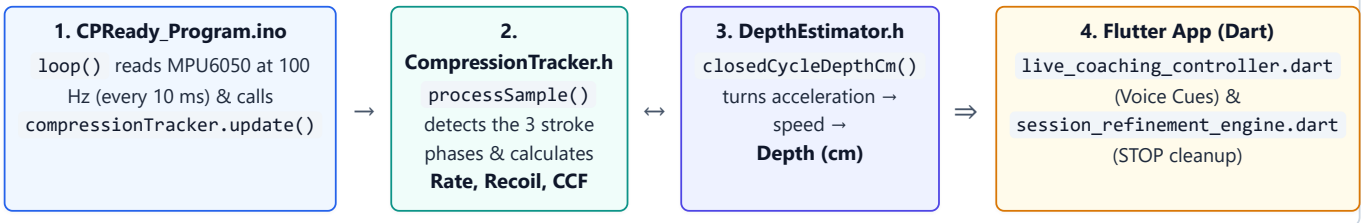
Question 8: Other conditions or factors that could affect single-MPU accuracy

> **15-SECOND DEFENSE ANSWER**
> "Aside from irregular rhythm, four physical factors can affect a single-MPU setup: (1) placing the manikin on a soft mattress instead of a hard floor, because the single sensor feels both the chest and the mattress sinking; (2) pushing off-center at an angle, which tilts the sensor mid-stroke; (3) loose vest straps that let the sensor pouch bounce; and (4) moving hands during the 2-second calibration step."

Physical Factor	Why It Affects a Single Chest MPU	How We Control It in Testing
1. Soft Surface Under Manikin ("Mattress Effect")	A single chest sensor measures downward travel relative to the room floor. On a soft mattress, pushing down compresses *both* the chest and the mattress underneath, inflating depth.	Place the manikin on a **firm floor or solid table** during all trials.
2. Off-Center Pushing / Chest Rocking	Pushing toward the side of the sternum tilts the sensor slightly (`3°–5°`) mid-stroke, slightly altering how much gravity projects onto the vertical axis (`~1–2 mm`).	Keep hands centered over the sternum pouch using standard CPR hand placement.
3. Loose Vest Straps / Shifting Pouch	If the vest is loose, lifting the hands at the top of a stroke lets the fabric pouch lift or slap independently of the plastic chest plate.	Fasten the vest straps snugly flat against the manikin torso before zeroing.
4. Moving During the 2-Second Zeroing Step	Pressing on the chest during the 2-second `CALIBRATING` screen would record a shaky "0 cm" baseline.	Firmware automatically rejects unsteady zeroing (`ERR_MOTION_UNSTABLE`) and prompts a retry.

Question 9: Basic Code Flow Walkthrough & Minor Troubleshooting Cheat Sheet

You only need to know **3 files on the ESP32** and **2 files on the Flutter App** to trace how sensor movement becomes Depth, Rate, Recoil, and CCF, or to make quick adjustments during testing:



Function-by-Function Code Walkthrough for Each CPR Indicator

1. **Where Every Reading Starts (100 times per second):**
   In `CPReady_Program/CPReady_Program.ino` → `loop()` reads the MPU6050 sensor every 10 ms, passes the X, Y, Z values to `compressionTracker.update(...)`, and sends the 16-byte Bluetooth packet via `bleManager.notifyReading(...)`.
2. **How DEPTH (`depth_cm`) Is Calculated in Code:**
   - In `CompressionTracker.h` → `processSample(...)` subtracts resting gravity (`computeProjectedAccelG`) and pushes each 10 ms reading into `depthEstimator.push(netAccel, dtSeconds)`.

- When a push finishes (`StrokeState::RECOILING` returns to rest), it calls `DepthEstimator::closedCycleDepthCm(...)` in `DepthEstimator.h`, which runs 3 short loops over that stroke's samples: (1) subtracts average acceleration `meanA`, (2) integrates acceleration into velocity `v += a * dt`, and (3) integrates velocity into position `x += (v - meanV) * dt`, returning `maxX - minX`.
- Finally, `CompressionTracker.h` multiplies by the saved Calibration Factor: `scaledDepthCm = physicalDepthCm * (depthScaleFactor / DEFAULT_DEPTH_SCALE)`.

3. **How RATE (`rate_cpm`) Is Calculated in Code:**
   - In `CompressionTracker.h` → inside `processSample(...)`, when a stroke finishes, it measures milliseconds since the previous stroke: `rawPeriodMs = currentMillis - lastStrokePeakMillis`.
   - If `rawPeriodMs` is between `300 ms` and `1500 ms`, it computes `instantRate = 60000.0f / rawPeriodMs` and calls `calculateRollingRate()` to average the last 4 strokes for the live display.
4. **How RECOIL (`recoil_complete` / `recoilPercentage`) Is Calculated in Code:**
   - In `CompressionTracker.h` → while in `StrokeState::RECOILING`, the code tracks the strongest upward rebound reading in `troughAccel`.
   - When the stroke completes, it checks: `lastStrokeRecoilComplete = (troughAccel < activeTriggers.recoilCompleteThresholdG)` (default `-0.35g`). If true, it increments `goodRecoilStrokeCount`.
5. **How CCF (`ccf_percent`) Is Calculated in Code:**
   - In `CompressionTracker.h` → `processSample(...)` anchors `firstCompressionStartMillis` on the first detected compression (crediting `550 ms`), matching the Laerdal manikin so pre-start waiting time on "*Start When Ready*" is excluded.
   - Every continuous stroke (≤ `1500 ms`, including sub-300 ms bounce fragments) adds `min(rawPeriodMs, 1000)` to `totalActiveCompressionMillis`, while any pause >1.5 s is left uncredited.
   - In `finalizeSession(...)`, CCF is computed over the full resuscitation window from Push #1 to `endMillis`: `(totalActiveCompressionMillis / activeSessionMs) * 100.0f`.

Minor Troubleshooting Cheat Sheet (Where to Change Numbers If Needed)

Almost all adjustable constants live in **one single file** on the ESP32: `CPReady_Program/include/CPReadyConfig.h` (and many can be changed directly in the mobile app without reflashing!):

If You Notice This During Testing...	Fastest App Fix (No Reflashing)	Exact Code Constant to Tweak (If Reflashing)
Depth is consistently 2–4 mm higher or lower than the manikin	Open App → **Manikin Calibration** (15s trial) or **Device Settings** → adjust **K Factor** (`±0.20` on *K* = `±1 mm` depth).	`CPReadyConfig.h` : `DEFAULT_MANIKIN_DEPTH_SCALE = 10.85f`
Very light/shallow pushes (~3.0 cm) are not being counted	Open App → set Sensitivity Mode to `HIGH`.	`CPReadyConfig.h` : Lower `MIN_START_THRESHOLD_G` (`0.08f` → `0.06f`) or `MIN_STROKE_DEPTH_CM` (`0.25f` → `0.20f`).
Accidental hand bumps during pauses get counted as strokes	Open App → set Sensitivity Mode to `STRICT`.	`CPReadyConfig.h` : Raise `MIN_START_THRESHOLD_G` (`0.08f` → `0.11f`) or `MIN_STROKE_DEPTH_CM` (`0.40f`).
Recoil % feels too strict (marks normal releases as incomplete)	—	`CPReadyConfig.h` : Change `RECOIL_COMPLETE_ACCEL_G` from `-0.35f` to `-0.25f` (closer to zero = more lenient).
CCF % drops too quickly during brief hands-off pauses	—	`CPReadyConfig.h` : Increase `PAUSE_THRESHOLD_MS` from `1500` (1.5 s) to `2000` (2.0 s).
Voice Coach speaks too often or not often enough	Tap the **Mute Speaker icon** on the Live Practice screen header.	`live_coaching_controller.dart` : Adjust `consecutiveErrorStrokes` (`5`) or `promptCooldownMs` (`5000` ms).

Part 3: Recommendations for Future Researchers (Chapter 5 of the Paper)

Below are **four well-justified recommendations** ready to include in Chapter 5 (*Recommendations / Future Work*), each explaining both **what to add** and **why it is recommended** based on our development findings:

1. Integrate a Thin Force-Sensitive Resistor (FSR) or Contact Switch Beneath the Palm Pad

Why It Is Recommended: An accelerometer can only sense *motion* (changes in speed), which means it cannot directly tell the difference between a trainee resting their hands lightly on the chest versus leaning 5 kg of body weight on the sternum at a standstill. Adding a paper-thin pressure sensor inside the palm pad solves two major challenges at once: it directly measures residual leaning weight for **true Chest Recoil verification**, and it provides an exact physical "push started / push ended" switch that eliminates integration window errors during irregular rhythm.

> **PAPER-READY RECOMMENDATION TEXT**
>
> *"Future iterations of the CPReady hardware should integrate a low-profile Force-Sensitive Resistor (FSR) or mechanical contact switch within the sternal compression pad. Because an accelerometer measures only kinematic acceleration and cannot directly observe static downward force at rest, incorporating a contact-pressure sensor will enable direct measurement of incomplete chest release (leaning force) and provide an exact mechanical zero-velocity boundary marker at the start and end of each compression."*

2. Utilize a Dual-Sensor Differential Setup When Operating on Soft Surfaces (Hospital Mattresses)

Why It Is Recommended: During standard floor-based or tabletop training on the Laerdal Little Anne QC, the back of the manikin cannot sink downward, making a single chest sensor optimal and reliable. However, in hospital bed resuscitation simulations performed on a soft foam mattress, pushing on the chest compresses both the patient's sternum and the mattress underneath; placing a second reference sensor on the manikin's backplate allows the system to subtract the mattress sinking distance so only true chest compression depth is reported.

> **PAPER-READY RECOMMENDATION TEXT**
>
> *"While a single sternal IMU provides optimal reliability and accuracy on rigid training surfaces, future studies evaluating CPR performance on compliant hospital mattresses should employ a second reference accelerometer mounted on the manikin backplate. Subtracting the vertical acceleration of the support surface from the sternal sensor in real time will prevent mattress deflection from inflating measured compression depth."*

3. Upgrade Sensor Communication to High-Speed SPI (200–400 Hz) with 6-Axis Gyroscope Tilt Compensation

Why It Is Recommended: Forceful chest compressions on a hinged manikin torso cause the sternum plate to rock slightly (`3°–5°`) during each push, and standard I2C jumper wires can occasionally experience brief timing jitter under repeated flexing. Upgrading to a modern SPI inertial sensor running at `200–400 Hz` and fusing the built-in 3-axis gyroscope will allow the microcontroller to continuously track the exact tilt angle of the chest mid-push, eliminating off-center tilt drift and making the sensor link immune to wire-flex pauses.

> **PAPER-READY RECOMMENDATION TEXT**
>
> *"Future hardware revisions should transition from a 100 Hz I2C accelerometer to a 200–400 Hz SPI 6-axis inertial measurement unit with continuous gyroscope-accelerometer sensor fusion. Tracking dynamic sternal tilt angles (3°–5°) throughout each compression stroke will eliminate gravity-projection errors during off-axis pushing while providing higher temporal resolution for stroke boundary detection."*

4. Expand Multi-Manikin Calibration Profiles (Adult, Child, and Infant Presets) and Per-Stroke Optical Reference Benchmarking

Why It Is Recommended: The present study validated CPReady against a single adult Laerdal Little Anne QC manikin, which only reports whole-session averages rounded to the nearest millimeter rather than individual stroke depths. Benchmarking the vest across multiple manikin models (including pediatric and infant compliances) alongside a continuous linear potentiometer or optical draw-wire sensor will allow future researchers to validate accuracy on every individual stroke and pre-load one-tap Manikin Profiles into the mobile app.

> **PAPER-READY RECOMMENDATION TEXT**
>
> *"Because the present evaluation was conducted on a single adult Laerdal Little Anne QC manikin that exports only session-level summary metrics, future researchers should validate the device across multiple manikin models (including pediatric and infant compliances) and instrument the test rig with a continuous linear potentiometer or optical draw-wire encoder. Capturing ground-truth displacement per stroke will enable granular waveform validation and support pre-configured calibration profiles in the mobile application."*